-- quick guide · 9 commands

SQL

SQL explained for beginners

The nine keywords that handle **90% of your day** with databases — and the gotchas nobody warns you about.

-- before the syntax

SQL is the skill that outlives frameworks

Frameworks come and go, but relational databases have run the world's data for 50 years. Learn to think in **sets** and the same model works everywhere.

- It's **declarative** — you say what you want, the engine finds how.
- Indexes and the optimizer do the heavy lifting, **if you let them**.

> WHY IT MATTERS
>
> Backend, analytics, ML — every data role runs on SQL. You rarely get to avoid it.

-- 01 · read data

Pick which columns you want

select.sql

```sql
SELECT * FROM users;
SELECT id, name FROM users;
```

GOTCHA

`SELECT *` ships every column and breaks when the schema changes. Name the columns you actually use in production.

-- 02 · filter rows

Keep only the rows that match

where.sql

```sql
SELECT * FROM users
WHERE country = 'MX'
  AND last_login IS NOT NULL;
```

GOTCHA · NULL

`= NULL` never matches — NULL means "unknown". Use `IS NULL`. And a function on the column blocks the index.

-- 03 · sort

Sort the result your way

order.sql

```sql
SELECT * FROM users
ORDER BY created_at DESC, id;
```

GOTCHA

Ties sort in an undefined order — add a tiebreaker like `id`. Sorting big results is costly unless the column is indexed.

-- 04 · paginate

Ask for just a few rows

limit.sql

```sql
SELECT * FROM users
ORDER BY id
LIMIT 20;
```

PRO TIP

`OFFSET n` re-scans n rows on every page. For deep pagination, filter by the last id (`WHERE id > ...`) instead.

-- 05 · combine tables

Glue two tables by a shared key

join.sql

```sql
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON o.user_id =
    u.id;
```

GOTCHA

INNER drops unmatched rows; LEFT keeps them as NULL. Many matches on the right **fan out** into duplicates — aggregate for one row per user.

-- 06 · aggregate

Collapse rows into one per group

group.sql

```sql
SELECT country, COUNT(*)
FROM users
GROUP BY country
HAVING COUNT(*) > 100;
```

GOTCHA

Every selected column must be aggregated or in GROUP BY. `WHERE` filters rows before grouping; `HAVING` filters groups after.

-- 07 · search text

Match text by a pattern

like.sql

```sql
SELECT * FROM users
WHERE name LIKE 'A%';
```

GOTCHA

A leading wildcard (`'%term'`) can't use an index and scans the whole table. For real search, reach for full-text or trigram indexes.

-- 08 · add data

Add new rows to a table

insert.sql

```sql
INSERT INTO users (name, email)
    VALUES
  ('Ada',  'ada@mail.com'),
  ('Alan', 'alan@mail.com');
```

PRO TIP

Insert many rows in one statement, not a loop — one round-trip instead of thousands.

-- 09 · change & remove

Edit or delete existing rows

write.sql

```sql
UPDATE users SET active = false
WHERE last_login < '2024-01-01';

DELETE FROM users WHERE id = 42;
```

GOTCHA

No `WHERE` means **every row**. Run your filter as a `SELECT` first to see exactly what you'll touch.

-- what trips people up

The mistakes that cost you hours

- `IS NULL` matches nulls — `= NULL` never does.
- No `WHERE` on an UPDATE or DELETE hits **every row**.
- `SELECT *` in production ships columns you don't need.
- Functions on indexed columns **quietly kill** performance.

> PRO TIP
>
> Read the plan with `EXPLAIN` before blaming the database. Most "slow SQL" is a missing index.

-- that's the 90%

You just learned the core of SQL

Master these nine and you can query almost any database. Which one bites you most often?

Alejandro Sánchez Yalí
Software Developer | AI & Blockchain Enthusiast
www.asanchezyali.com

study with me

the full source on GitHub